SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 50**
Question Count: 5
Duration : 60 Minutes

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Analytical Problem Solving

1. Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications. Analyze performance, security, and manageability, then justify the better choice.
2. A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware. Analyze how virtualization architecture supports isolation and identify two major attack surfaces.
3. Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s. Explain the likely bottlenecks.
4. Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter. Support your answer with compute, storage, and network virtualization considerations.
5. A multi-cloud federation project fails due to identity mismatches between providers. Analyze the issue and propose a secure identity and privacy design using federation concepts.

Equivalent Steps in VMware Workstation Pro

- Library pane showing multiple VMs = multiple instances/regions.
- VM → Snapshot → Take Snapshot, then Snapshot Manager = backup/versioning demonstration.
- Edit → Virtual Network Editor to show NAT/Host-only/Bridged networks = hybrid or multi-cloud connectivity.
- VM → Manage → Clone = replica/auto-scaling demonstration.

Tip: Paste each screenshot directly under the relevant question in this document (Insert → Picture in Word) before final submission, with a one-line caption explaining what it demonstrates.

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow

Total Marks Awarded:

Question 1: Type-1 versus Type-2 Hypervisors for Mission-Critical Applications

1.1 Problem Interpretation

A cloud datacenter hosting mission-critical applications (for example banking transactions, hospital information systems or e-governance portals) must deliver predictable performance, continuous availability, strong tenant isolation and centralised, automated operations. The task is to compare the two hypervisor classes against performance, security and manageability, and then select the one that satisfies all these requirements with the lowest operational risk.

1.2 Architectural Background

A **Type-1 (bare-metal) hypervisor** is installed directly on the server hardware and behaves as the operating system of the machine. Examples are VMware ESXi, Microsoft Hyper-V, Xen and KVM (the Linux kernel itself becomes the hypervisor). A **Type-2 (hosted) hypervisor** is installed as an application on top of a general-purpose host operating system such as Windows, Linux or macOS. Examples are VMware Workstation Pro, Oracle VirtualBox and Parallels Desktop. The comparison is summarised in Table 1.

Table 1: Comparison of Type-1 and Type-2 hypervisors

Criterion	Type-1 (Bare-metal)	Type-2 (Hosted)
Position in stack	Hardware, Hypervisor, VMs	Hardware, Host OS, Hypervisor, VMs
CPU and memory path	Direct scheduling on physical CPUs with hardware assistance (Intel VT-x / AMD-V); NUMA-aware	Requests pass through the host OS scheduler and memory manager and compete with host applications
I/O path	Optimised paravirtual drivers, SR-IOV, direct device assignment	Emulated devices routed through host OS drivers, adding context switches
Performance overhead	Low and predictable	Higher and variable
Attack surface	Small, purpose-built code base	Entire host OS, its services and user applications
Management	Centralised clustering, live migration, HA, load balancing, API automation	Per-host, largely manual; no native clustering or HA
Scalability	Dense consolidation; clusters of many hosts	A few VMs on a desktop-class machine
Typical use	Production datacenters, private and public clouds	Development, testing, training, demonstrations

1.3 Analysis

Performance. In a Type-1 design the hypervisor schedules vCPUs directly on physical cores, manages memory with hardware-assisted nested paging (EPT/NPT) and services storage and network I/O through lightweight paravirtual drivers. In a Type-2 design every guest I/O request travels from the guest OS, through the hypervisor, into the host OS and only then to the device driver. The host OS also runs its own processes, so guest performance fluctuates with host load, which is unacceptable for latency-sensitive transactional systems.

Security. The trusted computing base (TCB) of a Type-1 hypervisor is limited to the hypervisor and its management agents, which can be hardened, signed and measured through Secure Boot

and TPM attestation. In a Type-2 design the TCB includes the complete host operating system with browsers, drivers, remote-access tools and user accounts. A single compromise of the host (malware, a stolen credential, an unpatched service) exposes every VM running above it. Type-1 hypervisors are not immune, since VM-escape and side-channel flaws still occur, but the exposed surface is far smaller and easier to patch uniformly.

Manageability. Type-1 platforms are designed for fleet operations. Centralised managers (such as vCenter, System Center VMM or OpenStack) provide clustering, live migration, automatic restart of failed VMs (high availability), dynamic load balancing, templates, role-based access control, patch orchestration and REST APIs for automation. A Type-2 hypervisor is administered one machine at a time through a desktop interface, which cannot meet the service-level objectives of a datacenter.

1.4 Justification of the Better Choice

A **Type-1 hypervisor is the better choice** for the mission-critical datacenter because it (i) removes the host-OS layer and therefore gives near-native, predictable performance; (ii) shrinks the attack surface and supports hardware-rooted trust; and (iii) enables clustering, live migration and HA so that planned maintenance and hardware failures do not interrupt services. Its limitations (specialised hardware compatibility lists, licensing cost, and skilled administration) are small compared with the cost of downtime or a breach. Type-2 hypervisors remain appropriate for developer laptops, laboratories and proof-of-concept work such as the VMware Workstation Pro environment used in this assessment.

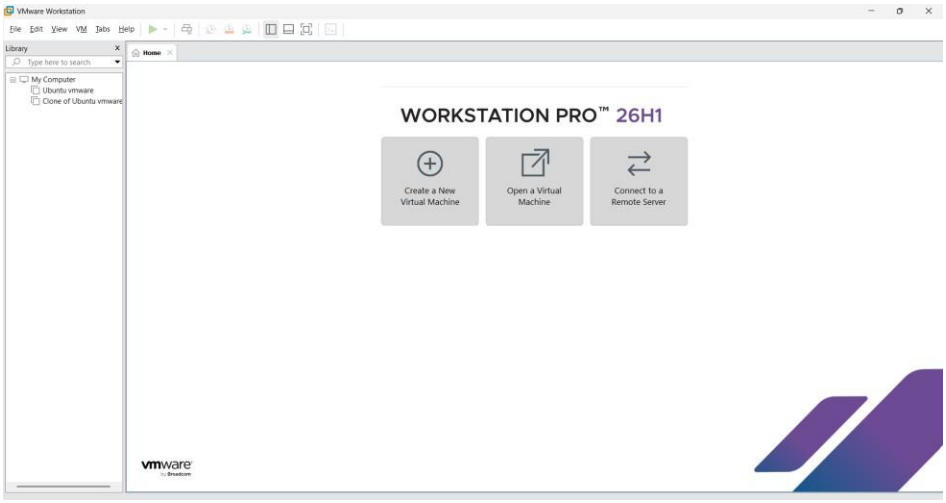


Figure 1: Library pane showing multiple VMs (Ubuntu vmware and its clone), representing multiple cloud instances / regions.

Question 2: Workload Isolation for Finance, Healthcare and Student Portals

2.1 Problem Interpretation

Three tenants with different sensitivity and compliance needs share the same physical servers: finance (payment and ledger data, PCI-DSS style controls), healthcare (patient records, HIPAA style controls) and student portals (academic and personal data, institutional privacy rules). The provider must ensure that a failure, misconfiguration or compromise in one tenant cannot affect the confidentiality, integrity or availability of the others.

2.2 How Virtualization Architecture Supports Isolation

The hypervisor sits between the hardware and every VM, so it mediates each access to CPU, memory, storage and network. Isolation is built in layers, as shown in Table 2.

Table 2: Isolation mechanisms in a virtualized datacenter

Layer	Isolation mechanism	Benefit
CPU	Hardware-assisted virtualization (VT-x / AMD-V) runs guests in a less-privileged mode; hypervisor traps privileged instructions	A guest kernel cannot control the physical CPU
Memory	Separate guest address spaces mapped by nested page tables (EPT/NPT); cross-VM page sharing disabled for sensitive tenants	No VM can read another VM's memory
I/O devices	IOMMU (Intel VT-d / AMD-Vi) restricts DMA; virtual NICs and virtual HBAs per VM	Prevents device-based memory snooping
Network	Separate virtual switches, port groups, VLAN / VXLAN segments, micro-segmentation firewalls	Tenant traffic is segregated and policy-controlled
Storage	Dedicated datastores or volumes, LUN masking, per-VM disk encryption with tenant-owned keys	Tenant data is unreadable by other tenants and by stolen media
Management	Role-based access control, separate resource pools, audit logs, dedicated management network	Limits administrator privileges and traces actions
Placement	Anti-affinity rules and dedicated clusters for the most sensitive workloads	Reduces blast radius and side-channel exposure

For this scenario, finance and healthcare would be placed on separate VLAN/VXLAN segments with micro-segmentation rules, encrypted virtual disks and, where regulation demands, dedicated hosts or clusters. The student portal can share a general-purpose cluster, but still in its own network segment and resource pool.

2.3 Two Major Attack Surfaces

Attack surface 1: The hypervisor layer (VM escape and side channels). Because all tenants rely on one hypervisor, a flaw in its emulated devices, drivers or memory handling can let code running in one guest break out and gain control of the host (for example the VENOM floppy-controller flaw, CVE-2015-3456). Co-residency also exposes micro-architectural side channels such as Spectre, Meltdown and L1TF, through which a malicious tenant may infer secrets of a neighbouring VM. **Mitigations:** prompt patching and microcode updates, removal of unused virtual devices, disabling cross-VM page sharing, dedicated hosts for high-risk tenants, Secure Boot with TPM attestation.

Attack surface 2: The management plane and virtual network. Management consoles, hypervisor APIs, snapshot and image repositories, and virtual switches are shared control points. A stolen administrator credential, an exposed API or a mis-assigned VLAN lets an attacker clone VMs, read snapshots (which contain memory and secrets), alter images or sniff traffic across tenants. VM sprawl and stale snapshots widen the exposure. **Mitigations:** multi-factor authentication and RBAC with least privilege, an isolated management network, signed and scanned images, encrypted snapshots, micro-segmentation and continuous logging with alerting.

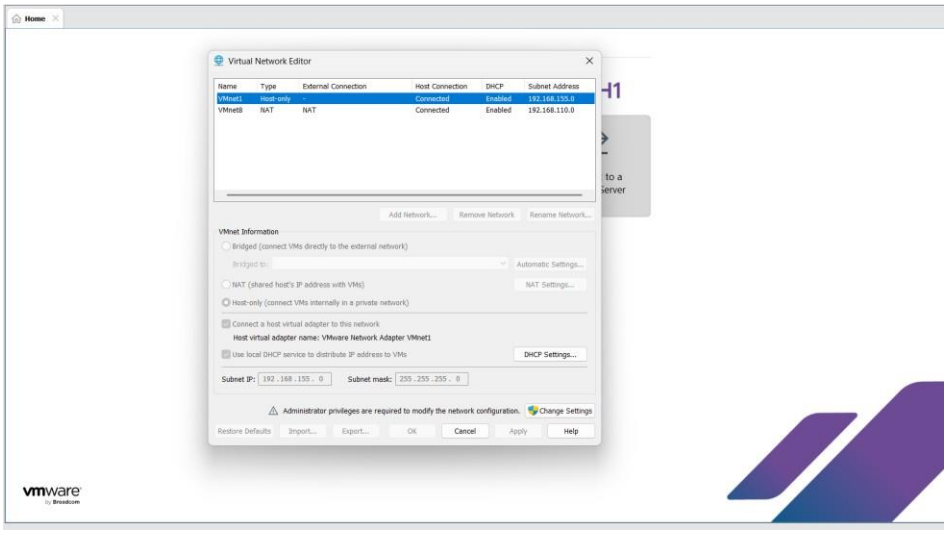


Figure 2: Virtual Network Editor showing separate Host-only (VMnet1) and NAT (VMnet8) segments used to isolate tenant networks.

Question 3: VM Host Utilization Analysis and Corrective Actions

3.1 Problem Interpretation

The snapshot shows a host that is almost saturated on three resources at once and is also slow in a user-visible operation (VM boot). The task is to identify which bottlenecks are most likely, explain how they interact, and recommend corrective actions.

Table 3: Observed metrics and interpretation

Metric	Observed	Typical guideline	Interpretation
CPU utilization	92%	Below about 70-80% sustained	Little headroom; vCPUs queue for physical cores (high CPU ready time), likely vCPU over-allocation
Memory utilization	88%	Below about 80-85%	Near the point where the hypervisor starts ballooning, compressing or swapping memory
Storage I/O latency	35 ms	Below about 10-20 ms	Disk queues are congested; the storage tier or path cannot serve the I/O demand
Average VM boot time	170 s	Roughly 30-90 s for a tuned VM	Symptom of the combined CPU, memory and storage contention, possibly a boot storm

3.2 Likely Bottlenecks and Cause-Effect Analysis

Primary bottleneck: storage I/O. A latency of 35 ms is well above what a healthy virtual datacenter should sustain. Booting a VM is dominated by many small, random reads of the operating system image, so high latency directly lengthens boot time. Several VMs booting together (a boot storm) or many VMs on the same datastore make the queue deeper.

Secondary bottleneck: memory pressure feeding the storage problem. At 88% usage, memory over-commitment may force the hypervisor to reclaim pages through ballooning, compression and finally host swapping. Swapping converts memory shortage into extra disk I/O, which raises the storage latency further. This is a vicious cycle: memory pressure increases I/O, I/O delays slow the guests, and slower guests hold resources longer.

Tertiary bottleneck: CPU contention. At 92% utilization, vCPUs wait in the scheduler queue before running. A VM with many vCPUs may also suffer co-scheduling delays (co-stop), which slows OS start-up, driver loading and service initialisation, adding directly to the 170 s boot time. Part of the CPU load may also come from the hypervisor itself handling interrupts and storage I/O.

In summary, the host is over-consolidated. The visible 170 s boot time is the combined result of CPU ready delay, memory reclamation and slow shared storage.

3.3 Recommended Corrective Actions

Immediate (minutes to hours):

- Identify noisy-neighbour VMs using hypervisor counters (CPU ready and co-stop, balloon and swap values, datastore latency) and throttle or relocate them.
- Live-migrate (vMotion or equivalent) selected VMs to under-utilised hosts to bring CPU and memory below about 75-80%.
- Stagger VM start-up with boot delays and startup order to avoid boot storms.

Short term (days):

- Right-size VMs: reduce unused vCPUs and memory, because over-sized VMs increase scheduling delay and memory pressure.
- Reduce memory over-commitment and set reservations for critical VMs so they are never swapped.
- Move the busiest VMs to faster storage (SSD/NVMe tier), balance datastores and verify multipathing and queue depth.

Long term (weeks to months):

- Add hosts or scale out the cluster, and enable automatic load balancing (for example DRS) with threshold alarms.
- Use linked or instant clones from optimised templates, host-side caching and thin-provisioned, de-duplicated storage to speed up boot.
- Implement capacity planning with continuous monitoring so that CPU, memory and latency alarms trigger before saturation.

Expected result: CPU and memory below about 75-80%, storage latency below about 10-15 ms and boot time brought down toward 60 seconds or less, which should be verified by re-measuring after each change.

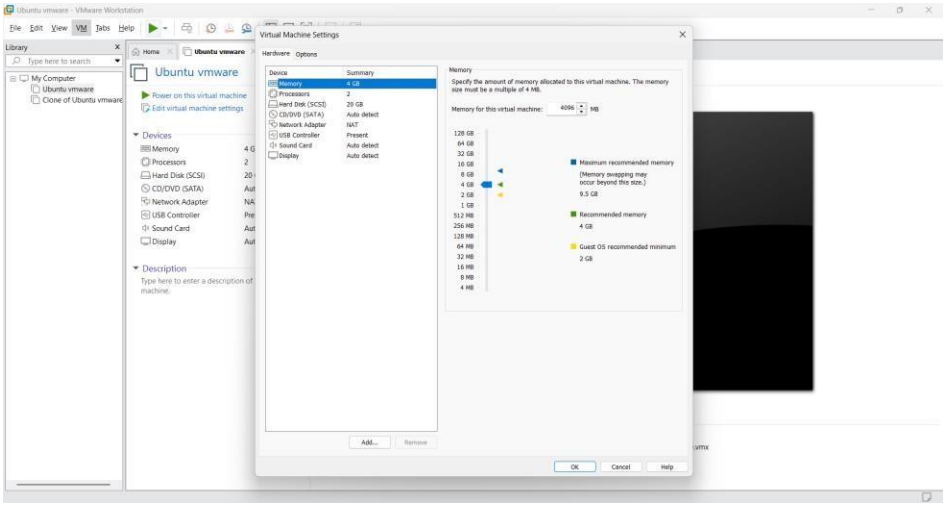


Figure 3: Virtual Machine Settings showing the memory and processor allocation that can be right-sized, with the clone listed in the Library for scale-out.

Question 4: Software Defined Datacenter versus Traditional Datacenter

4.1 Problem Interpretation

Agility is the ability of an organisation to provision, modify, scale and retire IT services quickly, consistently and at low cost in response to business demand. The task is to show why a Software Defined Datacenter (SDDC) is more agile than a traditional datacenter by examining compute, storage and network virtualization.

4.2 Concept

In a **traditional datacenter**, servers, storage arrays and network devices are purchased, racked and configured individually, usually in separate silos with separate teams and tools. In an **SDDC**, compute, storage, network and security are all virtualized, pooled and delivered as software services, controlled through policy-driven automation and exposed through APIs and self-service portals. Typical technology examples are vSphere or KVM (compute), vSAN or Ceph (storage) and NSX or Open vSwitch (network).

Table 4: Agility comparison

Aspect	Traditional datacenter	Software Defined Datacenter
Provisioning time	Days to weeks (procurement, cabling, manual set-up)	Minutes (templates, catalog, API calls)
Configuration	Manual, device by device; prone to drift and error	Policy-based and automated; consistent through code templates
Scaling	Over-provisioned hardware for peak demand	Elastic pooling and scale-out on demand
Utilization	Siloed, often low	Pooled resources, higher consolidation
Network change	VLAN and firewall changes on physical devices	Virtual networks and rules created in software
Resilience and DR	Hardware-specific replication, slow recovery	Snapshots, replication and automated failover at VM level
Management	Multiple consoles and teams	Unified control plane and orchestration

4.3 Contribution of Compute, Storage and Network Virtualization

Compute virtualization. Hypervisors abstract physical servers into a pool of CPU and memory, from which VMs are created from templates in minutes. Live migration allows maintenance without downtime, resource schedulers rebalance load automatically, and cloning enables rapid replication of application tiers. This removes the dependence of a workload on a particular physical server.

Storage virtualization. Physical disks and arrays are aggregated into logical pools. Thin provisioning allocates capacity only when used, policy-based management assigns performance and protection levels per VM, and software-defined or hyper-converged storage scales simply by adding nodes. Snapshots and replication provide quick versioning and recovery, so storage is no longer a delay in application delivery.

Network virtualization. Overlay technologies such as VXLAN or GENEVE, virtual switches, distributed routers and distributed firewalls create complete logical networks independent of the physical switches. A new isolated network, load balancer or security policy can be deployed by API in seconds, and micro-segmentation applies security at the VM level, travelling with the workload when it moves.

Automation layer. The orchestration and automation tools tie the three layers together. Infrastructure-as-code, self-service catalogs and policy engines allow a complete application environment (VMs, networks, storage and security rules) to be deployed repeatably, which is the core of agility.

4.4 Considerations

The SDDC introduces overlay and underlay troubleshooting complexity, licensing cost, a requirement for automation skills and some vendor dependence. These are outweighed by the gains in speed, consistency and utilization, provided governance, capacity planning and monitoring are in place.

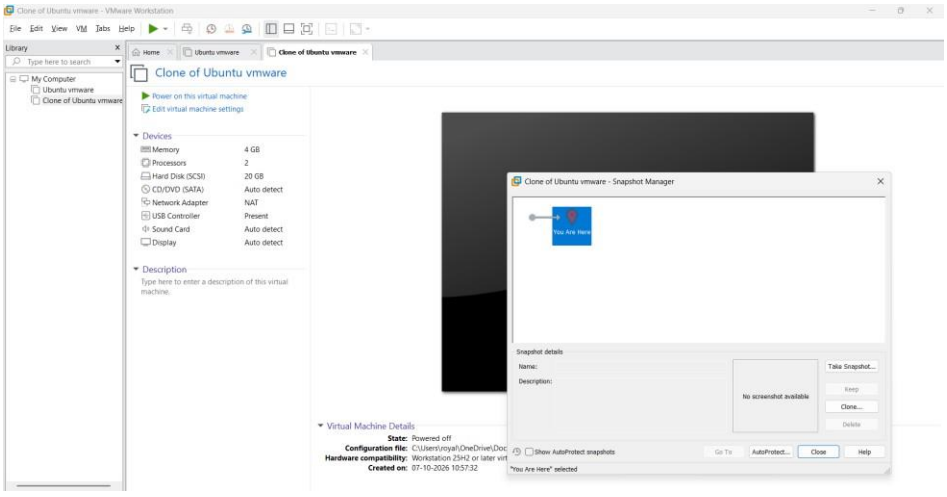


Figure 4: Snapshot Manager of the cloned VM, demonstrating versioning / backup and rapid replica creation.

Question 5: Identity Mismatch in a Multi-Cloud Federation

5.1 Problem Interpretation

A multi-cloud federation joins services from different providers so that users and applications can work across them. The project fails because each provider recognises users differently. The

task is to analyse why the identities do not match and to design a secure, privacy-preserving federated identity solution.

5.2 Analysis of the Issue

Table 5: Causes of identity mismatch and their effects

Cause	Effect	Remedy in the proposed design
Separate identity stores and namespaces in every provider (local accounts and different user ID formats)	Same person has multiple unlinked identities; duplicate or orphaned accounts	One authoritative identity provider (IdP) as the single source of truth
Different protocols and token formats (SAML, OIDC/OAuth 2.0, proprietary tokens)	Tokens from one provider are not accepted by another	Standards-based federation (SAML 2.0 / OpenID Connect) with an identity broker for translation
Inconsistent attributes and roles (different claim names, NameID formats, role models)	Authorization fails or users get wrong permissions	Claim and attribute mapping from IdP groups to local roles
No established trust (unexchanged metadata, expired certificates, clock skew)	Assertion signature or time validation fails	Metadata exchange, certificate rotation and time synchronisation
Unsynchronised user lifecycle	Leavers keep access; new users wait for access	Automated provisioning and de-provisioning with SCIM
Static keys and passwords between clouds	Credential leakage and large blast radius	Short-lived tokens and workload identity federation

5.3 Proposed Secure Identity and Privacy Design

The design uses core **federation concepts**: an Identity Provider (IdP) that authenticates users and issues signed assertions or tokens; Service Providers (SPs) or relying parties (each cloud) that trust the IdP; a trust relationship based on exchanged metadata and signing certificates; claims that carry identity attributes; and single sign-on (SSO) so that the user authenticates once.

Design components:

- **Central IdP and trust anchor:** a single enterprise IdP (for example Microsoft Entra ID, Keycloak or Okta) holds the authoritative identities. Every cloud is configured as a relying party using SAML 2.0 or OpenID Connect, and an identity broker translates between protocols where a provider supports only one.
- **Common identity schema:** a stable, unique and immutable identifier (not a reusable e-mail address) with a standard attribute set. Claim mapping translates directory groups to roles in each cloud (RBAC), with attribute-based rules for finer control.
- **Strong authentication:** multi-factor authentication and conditional access (device state, location, risk score) enforced at the IdP.
- **Lifecycle management:** SCIM-based provisioning and automatic de-provisioning, plus periodic access reviews.
- **Workload identity:** applications and services use short-lived OIDC tokens exchanged for temporary cloud credentials instead of long-lived keys.

- **Token security:** signed (and, where needed, encrypted) assertions, short lifetimes, audience restriction, replay protection, certificate rotation and synchronised clocks (NTP).
- **Privacy by design:** data minimisation so that each provider receives only the claims it needs; pairwise pseudonymous identifiers so that providers cannot correlate a user across clouds; user consent for optional attributes; purpose limitation, encryption in transit and at rest, and retention limits in line with applicable data-protection law (for example GDPR and India's DPDP Act, 2023).
- **Monitoring and audit:** federated login and administrative events are sent to a central SIEM so that cross-cloud activity can be traced.

Authentication flow:

1. The user requests a service in Cloud B (the relying party).
2. Cloud B redirects the user to the central IdP with an authentication request.
3. The IdP authenticates the user with MFA and evaluates conditional-access policy.
4. The IdP issues a signed assertion or ID token containing only the minimum required claims and a pseudonymous identifier.
5. Cloud B validates the signature, issuer, audience and expiry, maps the claims to a local role and issues a short-lived session or credential.
6. All events are logged and forwarded to the central SIEM for monitoring and audit.

Through this design the identity mismatch is removed at its source: one authoritative identity, one trust model and one policy set are projected consistently onto all providers, while exposure of personal data and credentials is minimised.

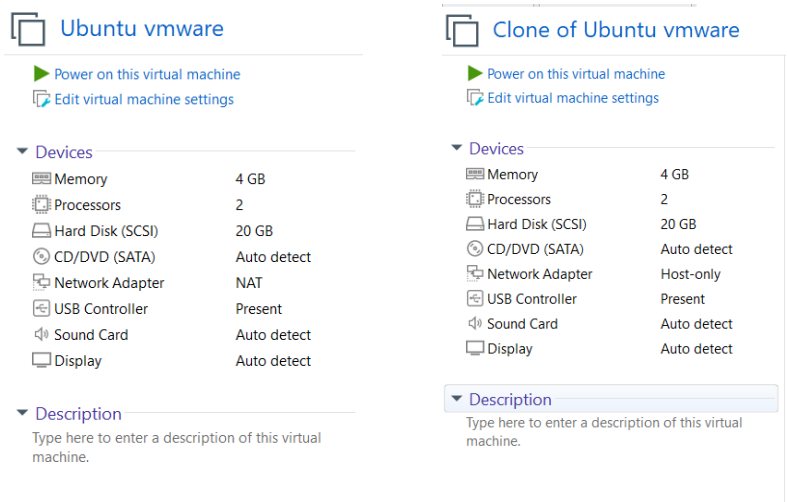


Figure 5: Ubuntu vmware on a NAT network (left) and Clone of Ubuntu vmware on a Host-only network (right), representing hybrid / multi-cloud connectivity.

Conclusion

The analysis shows that virtualization architecture decisions directly determine performance, security and agility. A Type-1 hypervisor suits mission-critical production, layered isolation with careful protection of the hypervisor and management plane keeps tenants separate, systematic monitoring and right-sizing resolve resource bottlenecks, SDDC principles deliver agility across compute, storage and network, and a federated identity design with privacy controls makes multi-cloud collaboration secure and manageable.

Table 6: Summary of key findings

Question	Key finding	Recommended decision
Q1	Type-2 adds a host-OS layer that reduces performance, enlarges the attack surface and lacks clustering	Adopt a Type-1 hypervisor for production
Q2	Isolation is layered (CPU, memory, I/O, network, storage, management); the hypervisor and the management plane are the main attack surfaces	Segment tenants, harden and patch the hypervisor, secure the management plane
Q3	Storage latency, memory pressure and CPU contention combine to slow VM boot	Rebalance and right-size VMs, upgrade storage tier, stagger boots, add capacity
Q4	Virtualized compute, storage and network with automation cut provisioning from weeks to minutes	Move to an SDDC with policy-driven orchestration
Q5	Separate identity stores, protocols and attribute models cause the federation failure	Central IdP with SAML/OIDC, claim mapping, SCIM, MFA and privacy-preserving claims